Face-Based Attendance System

Final Term Prototype Project Document, Integrative Programming and Technologies

1. Project Overview

1.1 The problem I am solving

Manual attendance (paper sheets, roll calls) is slow, easy to falsify because someone can sign for a classmate, and tedious to summarize. I want a system where a person looks into a webcam and the attendance is recorded automatically, with rules that prevent duplicates and mark late arrivals.

1.2 What I am building

A prototype where the webcam captures a face, a computer-vision library (Geitgey, 2020; King, 2009) detects it and converts it into a numeric encoding, my code compares that encoding with the ones stored in a database, applies attendance rules (match threshold, duplicate window, Present/Late cutoff), saves the record, and shows reports with a CSV export.

1.3 Why this fits the final-term requirements

My guide grades the **integration process**, not how advanced the AI is. My prototype connects several components (webcam, UI, validation module, vision library, rules engine, SQLite database, reporting module). Data visibly flows from input to output, it is validated and transformed, and the result is stored and delivered. I use a pretrained recognition model as one component, and I put my effort into the pipeline around it.

1.4 Scope and limits

- This is a solo prototype. I will enroll only myself and classmates who consented.
- It is not a security or identity-verification product. A printed photo or phone screen may fool it (no liveness detection). I will document this as a known limitation.

---

2. My Hardware

I ran inxi on my laptop to get the exact specifications. Here is what I have and what each part means for the project.

Component	My specification	What it means for my project
Laptop	ASUS VivoBook X415EA (X1400EA), UEFI X415EA.311	A portable demo machine; everything runs locally, no internet needed during the demo

Component	My specification	What it means for my project
CPU	Intel Core i3-1115G4 (Tiger Lake), 2 cores / 4 threads, 400 MHz to 4.1 GHz, AVX2 supported	Enough for CPU-based face detection (dlib HOG). AVX2 helps dlib run faster. Only 2 physical cores, so I should keep processing light
RAM	8 GiB total, about 7.45 GiB available at the time of the scan	Enough for Streamlit + dlib + SQLite (roughly 1 GB). I will close extra browser tabs while demoing
GPU	Intel UHD Graphics G4 (integrated, driver i915/iris)	No NVIDIA GPU, so no CUDA. I will use dlib's CPU **HOG** detector, not the CNN detector
Webcam	IMC Networks USB2.0 HD UVC WebCam (ID 13d3:5498), driver uvcvideo, USB 2.0 (480 Mb/s)	A standard UVC camera, so OpenCV and the browser can use it without extra drivers. USB 2.0 bandwidth is fine for 720p
Display	1366x768, 14-inch class (ChiMei InnoLux eDP-1), 60 Hz	A small screen. I will design the Streamlit pages compactly and avoid wide layouts
Storage	WD PC SN530 NVMe 238 GiB (root: 233 GiB, 120 GiB used) plus PNY CS1311 240 GB SATA SSD	About 113 GiB free on root, far more than the project needs (under 2 GB). I will keep backups on the second SSD (/dev/sda)
Swap	2 GiB swapfile	Fine, but compiling dlib is memory-hungry, so I will close other apps during installation
Battery	55.1% of design capacity (20.5 / 37.1 Wh), 1526 cycles	Weak. I must **keep the charger plugged in** for the dlib build and during the live demo
Thermals	CPU reading 72 C at the time of the scan	The dlib compile will push the CPU hard. I will keep the laptop on a hard surface with the vents clear
Network	Realtek RTL8821CE Wi-Fi	Only needed to download packages. The demo itself works offline
Other	Bluetooth, PipeWire audio, USB mouse	Not used by the project

Performance expectation. On this dual-core CPU, detection plus encoding may take roughly 0.3 to 1 second per frame at 640x480. I will measure it myself and report the real number. Downscaling the frame before detection makes it faster.

---

3. My Software Environment

3.1 What I already have

Item	Value	Why it matters
OS	Linux Mint 22.3 "Zena" (Ubuntu 24.04 "noble" base)	I can install the C++ toolchain with apt, so building dlib is straightforward
Desktop	Cinnamon 6.6.9 on X11	X11 works well with webcam apps
Kernel / compiler	7.0.0-31-generic, gcc 13.3.0	gcc is already present for building dlib

Item	Value	Why it matters
Python	System Python **3.12**	Newer than the 3.10/3.11 I would normally pick, so there are two things to handle (see 3.3)
Editor	VS Code (already installed from the Microsoft repo)	My development environment
Browser	Google Chrome (installed)	Streamlit's UI runs in it and it asks permission to use the webcam
Package manager	apt (2187 packages) and flatpak	System libraries come from apt, Python libraries from pip inside a venv

3.2 My technology stack

Layer	Tool	Role in the project
Language	Python 3.12	Every module is written in Python
Environment	venv	Isolated dependencies, and Ubuntu 24.04 blocks system-wide pip anyway
Image capture and processing	opencv-python (Bradski, 2000; OpenCV, n.d.)	Resize frames, color conversion, blur and brightness checks
Face detection and encoding	face_recognition (built on dlib)	HOG face detector (Dalal & Triggs, 2005) plus a ResNet-based model (He et al., 2016) that outputs a 128-number encoding per face
Numerics	numpy (Harris et al., 2020)	Encodings are numpy arrays; distance calculations
User interface	streamlit (Streamlit Inc., n.d.)	Multi-page web UI. st.camera_input captures a photo from my webcam
Database	SQLite (Hipp, n.d.) via Python's built-in sqlite3 module (Python Software Foundation, n.d.)	Stores people, face encodings, and attendance records in a single file
Data processing	pandas (McKinney, 2010)	Filtering, daily summaries, Absent calculation, CSV export
Charts	Streamlit charts or plotly	Present vs Late per day
Testing	pytest	Automated test cases for the 04_Testing folder
Version control	git	History of my work and proof of my individual contribution
Camera tools	v4l-utils, cheese	Verify the webcam works before I write code

Fallback plan. If dlib refuses to build, I will switch only the vision module to OpenCV's YuNet (detector) and SFace (recognizer). These are two small ONNX model files with no compilation. The rest of the architecture stays the same.

3.3 Installation steps on my machine

Step 1: system packages (Mint 22 / Ubuntu 24.04 package names)

```
sudo apt update
sudo apt install -y build-essential cmake python3-dev python3-venv \
   libopenblas-dev liblapack-dev libx11-dev libgtk-3-dev \
   v4l-utils cheese sqlite3
```

Step 2: check that the webcam works first

```
ls /dev/video*              # should list /dev/video0 (and usually video1)
v4l2-ctl --list-devices     # should show the USB2.0 HD UVC WebCam
v4l2-ctl --list-formats-ext # shows the resolutions it supports
groups                      # I should be in the 'video' group
cheese                      # visual test
```

If I am not in the video group: sudo usermod -aG video $USER, then log out and back in.

Step 3: create the project and the virtual environment

```
mkdir -p ~/face_attendance && cd ~/face_attendance
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
```

Step 4: install the libraries

```
pip install "setuptools<81" wheel
pip install numpy opencv-python pandas streamlit plotly pytest
pip install dlib          # compiles C++; plug in the charger and wait
pip install face_recognition
```

Step 5: verify

```
python -c "import face_recognition, cv2, streamlit, numpy; print('OK')"
```

3.4 Two things I need to watch because I have Python 3.12

1. **setuptools.** The face_recognition_models package still imports pkg_resources, which newer setuptools versions removed. A fresh venv on Python 3.12 does not include setuptools, so I install "setuptools<81" first, as in Step 4. If I see ModuleNotFoundError: No module named 'pkg_resources', this is the cause.
2. **numpy versions.** If I get an error mentioning NumPy binary compatibility when importing dlib or cv2, I run pip install "numpy<2" and reinstall opencv-python.

3.5 Compiling dlib on a dual-core laptop

The pip install dlib step compiles a large C++ library and can take 10 to 20 minutes on my i3. This is normal. Because my battery health is low and my CPU already runs warm, I will:

- keep the charger plugged in,
- close Chrome tabs and other heavy apps,
- put the laptop on a hard surface so the vents stay clear,
- not interrupt the build.

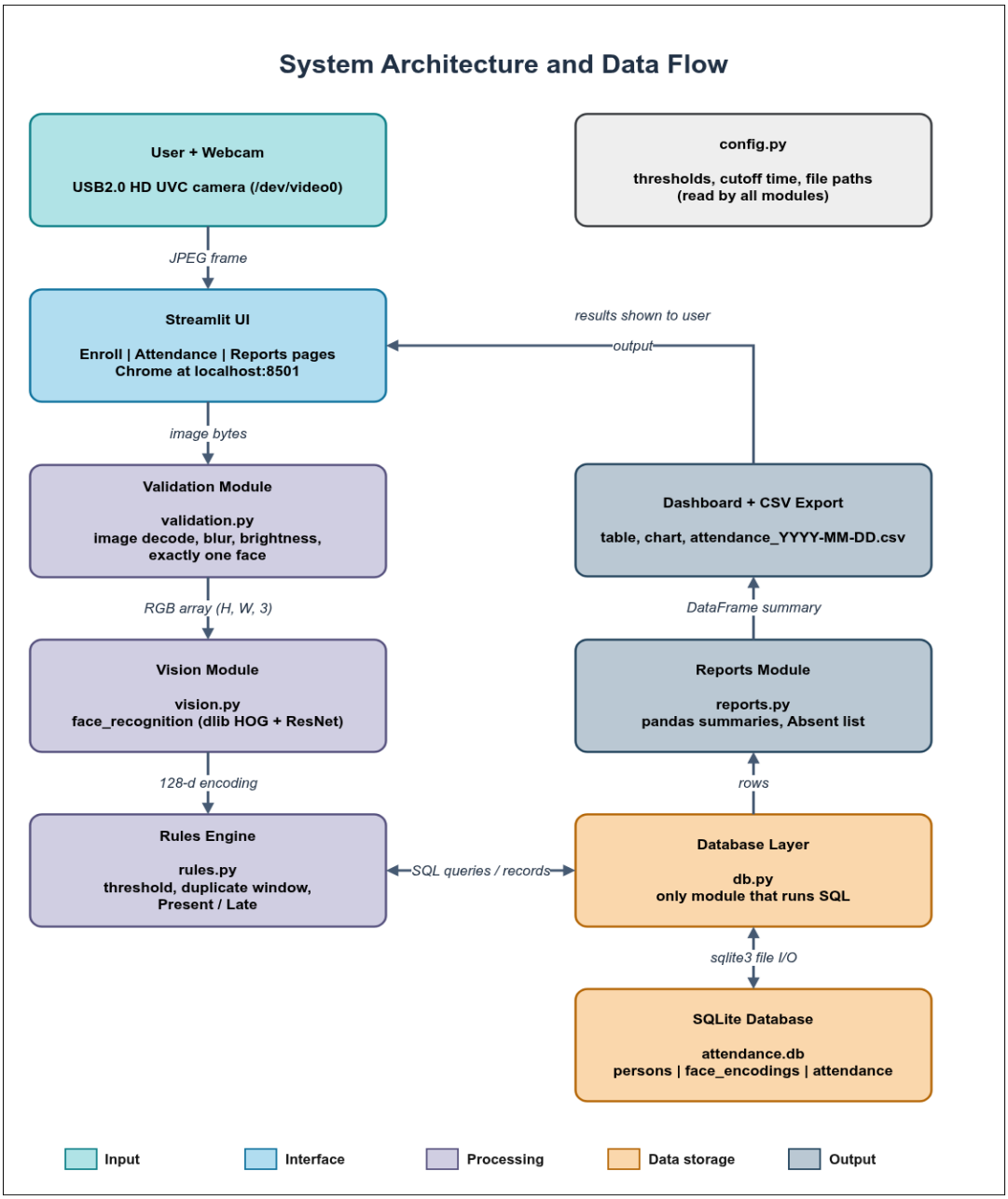
---

4. System Architecture

4.1 Data-flow diagram

Figure 1

System Architecture and Data Flow of the Face-Based Attendance System



Note. Arrows show the direction of data. Labels give the data format exchanged between components. All components run locally on my laptop.

4.2 Input-Process-Output (IPO) model

The IPO model summarizes what goes into my system, what happens to it, and what comes out. I apply it to each of the three main functions of the prototype. The SQLite database sits underneath all three as the storage layer, because enrollment writes to it, attendance reads from it and writes to it, and reporting reads from it.

Figure 2

Input-Process-Output (IPO) Model of the Face-Based Attendance System

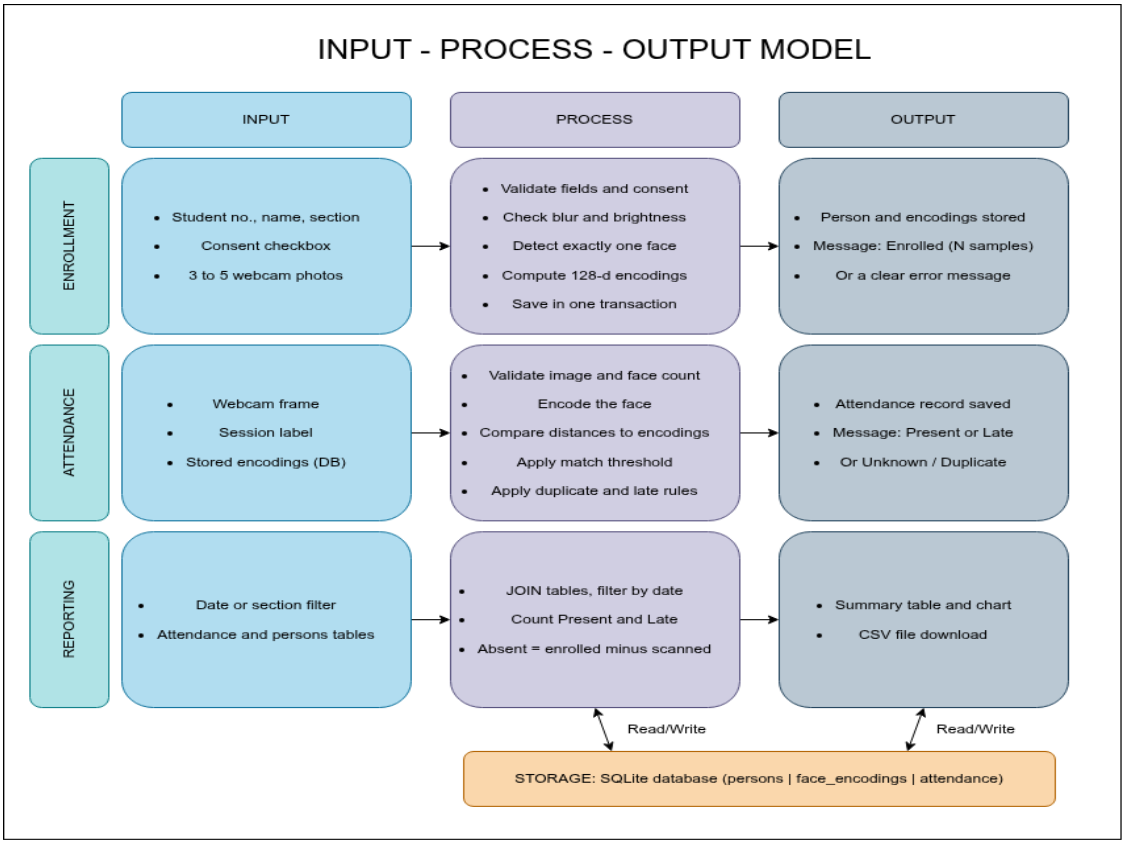


Table 1

IPO Summary of the Prototype

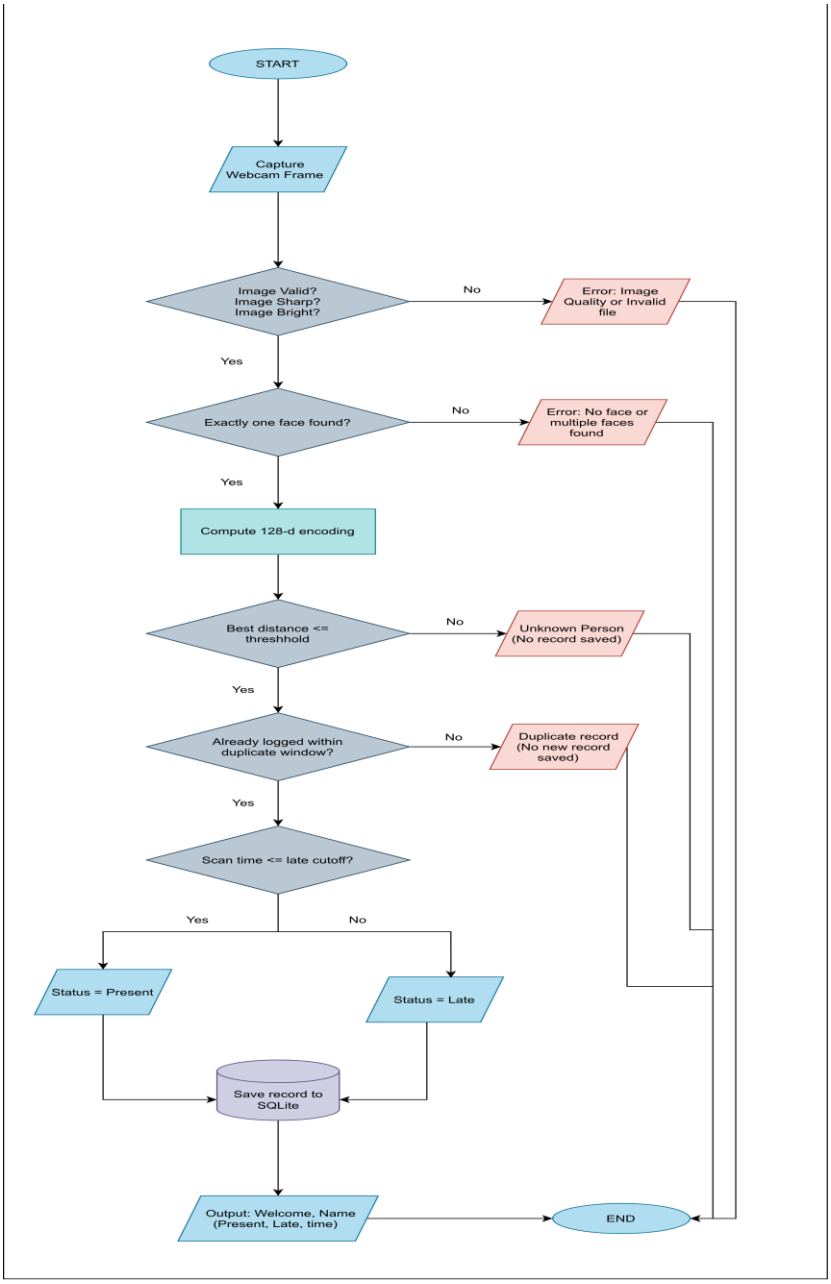
Function	Input	Process	Output
Enrollment	Student number, name, section, consent, 3 to 5 webcam photos	Validate fields and consent; check image quality; detect exactly one face; compute 128-d encodings; save in one transaction	Person and encodings stored in SQLite; confirmation or error message
Attendance	Webcam frame, session label, stored encodings	Validate image; detect and encode face; compare distances; apply match threshold, duplicate window, and late cutoff	Attendance record saved; Present, Late, Unknown, or Duplicate message
Reporting	Date or section filter; attendance and persons tables	Join tables; filter; count Present and Late; compute Absent (enrolled minus scanned)	Summary table, chart, and CSV file

4.3 Attendance process flowchart

The IPO model shows the overall flow, but the attendance function contains several decisions. The flowchart below shows each decision point and what happens on every path, including the error paths.

Figure 3

Flowchart of the Attendance Process



4.4 Components and responsibilities

Component	File	Responsibility
UI layer	app.py, pages/	Collects input (camera, forms) and shows results and errors
Validation	validation.py	Checks the image decodes, is bright and sharp enough, and contains exactly one face; validates form fields
Vision module	vision.py	Detects faces, computes encodings, calculates distances
Database layer	db.py	The only module that runs SQL: creates tables, inserts, queries
Rules engine	rules.py	Match threshold, duplicate window, Present/Late cutoff
Reporting	reports.py	pandas summaries, Absent list, CSV export
Configuration	config.py	Thresholds, paths, cutoff time in one place

Why I separated them this way. Each module can be tested alone, I can swap the vision library (dlib to SFace) without touching the UI or the database, and I can build and debug one module at a time, which suits working alone.

---

5. Database Design: How I Store Faces

5.1 The core idea

I do **not** store photos to do recognition. The library converts a face into an **encoding**, a vector of **128 floating-point numbers**. Two photos of the same person give vectors that are close together, and different people give vectors that are far apart. I store that vector in SQLite as a **BLOB** (binary data) and compare later.

- 128 values x 8 bytes (float64) = **1,024 bytes** per encoding
- 5 encodings for each of 30 people is only about 150 KB

5.2 Schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS persons (
    person_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    student_no    TEXT NOT NULL UNIQUE,
    full_name     TEXT NOT NULL,
    section       TEXT,
    consent_given INTEGER NOT NULL CHECK (consent_given IN (0,1)),
    consent_date  TEXT NOT NULL,          -- ISO 8601
    created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);
```

```
CREATE TABLE IF NOT EXISTS face_encodings (
   encoding_id INTEGER PRIMARY KEY AUTOINCREMENT,
   person_id   INTEGER NOT NULL REFERENCES persons(person_id) ON DELETE CASCADE,
   encoding    BLOB NOT NULL,          -- 128 float64 values = 1024 bytes
   created_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS attendance (
   attendance_id INTEGER PRIMARY KEY AUTOINCREMENT,
   person_id     INTEGER NOT NULL REFERENCES persons(person_id),
   scanned_at    TEXT NOT NULL,        -- ISO 8601 timestamp
   status        TEXT NOT NULL CHECK (status IN ('Present','Late')),
   distance      REAL NOT NULL,        -- match distance (lower = better)
   session_label TEXT                  -- e.g. 'IPT-Lecture-2026-10-05'
);

CREATE INDEX IF NOT EXISTS idx_att_person_time
   ON attendance(person_id, scanned_at);
```

PRAGMA foreign_keys = ON; must be run on every new connection, otherwise SQLite ignores the foreign keys.

5.3 Converting a numpy array to a BLOB and back

```
import numpy as np, sqlite3

# SAVE: numpy array -> bytes
blob = encoding.astype(np.float64).tobytes()
conn.execute("INSERT INTO face_encodings(person_id, encoding) VALUES (?, ?)",
             (person_id, blob))

# LOAD: bytes -> numpy array
row = conn.execute("SELECT encoding FROM face_encodings WHERE person_id=?",
                   (person_id,)).fetchone()
encoding = np.frombuffer(row[0], dtype=np.float64)   # shape (128,)
```

I always check len(blob) == 1024 before loading. A different length means corrupt data, and the system shows a clear error instead of crashing.

5.4 Matching against the database

```
known = load_all_encodings()   # list of (person_id, np.array)
distances = face_recognition.face_distance([e for _, e in known], new_encoding)
best = int(np.argmin(distances))
if distances[best] <= MATCH_THRESHOLD:   # 0.5 strict, 0.6 library default
   person_id = known[best][0]
else:
   pass  # "Unknown person"
```

For a class-sized dataset, a linear scan is instant, so I do not need a vector database. In my defense I will mention FAISS as a scale-up improvement.

6. Data Flow in Detail

6.1 Enrollment flow

1. I open the **Enroll** page and enter student number, name, and section.
2. Validation checks required fields, student number format, no duplicate student number, and that the **consent checkbox** is ticked.
3. I capture 3 to 5 photos (slightly different angles and expressions).
4. For each photo: decode, check brightness and blur, detect faces. It is **rejected** if there are 0 faces or more than 1.
5. The vision module computes a 128-d encoding for each valid photo.
6. The database layer saves the person and all encodings in **one transaction** (all or nothing).
7. The UI confirms: "Enrolled Juan Dela Cruz with 4 face samples."

6.2 Attendance flow

1. I open the **Attendance** page and confirm the session label.
2. The webcam captures a frame (st.camera_input).
3. Validation: decodable image, not too dark, not too blurry, exactly one face.
4. The vision module encodes the face and computes distances to all stored encodings.
5. The rules engine decides:
   - best distance above the threshold: **Unknown person**, no record;
   - same person already logged within the duplicate window (for example 30 minutes, same session): **Duplicate**, no new record;
   - otherwise **Present** if the time is on or before the cutoff, else **Late**.
6. The database layer inserts the record, and the UI shows "Welcome, Juan: Present (09:02)".

6.3 Reporting flow

attendance JOIN persons becomes a pandas DataFrame, which is filtered by date or section, summarized (Present, Late, Absent), shown as a table and chart, and offered as a CSV download.

Absent calculation (a meaningful transformation). The set of enrolled persons minus the set of persons who have a record for that session.

---

7. Data Formats Exchanged

Between	Format
Webcam to UI	JPEG bytes (UploadedFile)
UI to Validation / Vision	numpy.ndarray, shape (H, W, 3), RGB, uint8
Vision to Rules	numpy.ndarray, shape (128,), float64, plus face location

Between	Format
	(top, right, bottom, left)
Rules to / from Database	SQL parameters and rows; encodings as 1024-byte BLOBs
Database to Reports	pandas DataFrame
Reports to user	On-screen table and chart, and CSV file attendance_YYYY-MM-DD.csv

8. Processing Rules and Transformations

Rule	Detail
Image to encoding	Raw pixels become a 128-d vector (the core transformation)
Distance threshold	Converts a distance into match / no match (0.5 to 0.6, which I will tune with my own data)
Duplicate window	Ignores repeated scans of the same person in the same session within N minutes
Late cutoff	Compares the scan time with the class start time plus a grace period
Aggregation	Per-person and per-date counts of Present, Late, and Absent

9. Validation and Error Handling

Situation	System response shown to the user
Camera unavailable or permission denied	“Camera not accessible. Check the connection and browser permission.”
File is not an image or is corrupt	“Invalid image file. Please capture a new photo.”
Image too dark or blurry	“Image quality too low. Improve lighting and hold still.”
No face detected	“No face found. Face the camera directly.”
Multiple faces	“Multiple faces detected. Only one person at a time.”
Unknown person	“Face not recognized. Please enroll first or try again.”
Duplicate scan	“Already recorded at 09:02 for this session.”
Missing consent	“Consent is required to enroll.”
Duplicate student number	“This student number is already enrolled.”
Database locked or missing	“Database unavailable. Try again.” (technical detail goes to a log file; the app never crashes)
Corrupt encoding (not 1024 bytes)	Skip that sample, log a warning, notify the admin

Quality checks with OpenCV:

- Blur: variance of the Laplacian, cv2.Laplacian(gray, cv2.CV_64F).var(). Below roughly 50 to 100 means blurry (I will tune it with my webcam).
- Brightness: gray.mean(). Below about 50 means too dark.

Performance tweak for my dual-core CPU. Before detection I downscale the frame (for example to 50%), find the face, and then compute the encoding. This keeps each scan responsive.

---

10. Project Folder Structure

This layout matches the submission package required by my guide.

```
face_attendance/
├── 01_Source_Code/
│   ├── app.py
│   ├── pages/
│   │   ├── 1_Enroll.py
│   │   ├── 2_Attendance.py
│   │   └── 3_Reports.py
│   ├── config.py
│   ├── validation.py
│   ├── vision.py
│   ├── db.py
│   ├── rules.py
│   ├── reports.py
│   └── requirements.txt
├── 02_Data/
│   ├── attendance.db          (sample DB with demo data)
│   ├── schema.sql
│   └── sample_images/         (valid and deliberately bad images)
├── 03_Documentation/
├── 04_Testing/
│   ├── test_cases (PDF/XLSX)
│   ├── bug_log
│   └── tests/                 (pytest files)
├── 05_Presentation/
├── 06_Demo_Backup/
├── README.md
└── Contribution_Record
```

I will run the app with `streamlit run 01_Source_Code/app.py`, and it opens in Chrome at `http://localhost:8501`. Chrome allows camera access on `localhost`.

---

11. Testing Plan

ID	Test	Expected result
T01	Enroll with a clear face and consent	Saved, 3+ encodings stored
T02	Enroll without consent	Rejected
T03	Enroll with a duplicate student number	Rejected

ID	Test	Expected result
T04	Photo with no face (wall, hand over lens)	"No face found"
T05	Photo with 2 faces	"Multiple faces"
T06	Dark and blurry images	Quality error
T07	PDF renamed to .jpg	"Invalid image"
T08	Enrolled person scans	Recognized, Present
T09	Same person scans again within 30 minutes	Duplicate message, no new row
T10	Scan after the cutoff time	Status = Late
T11	Unenrolled person scans	Unknown
T12	Photo of a printed photo or phone screen	Record the result; document the limitation
T13	Database file renamed or locked	Friendly error, no crash
T14	Report for a date with no data	Empty-state message
T15	CSV export	File opens with the correct columns
T16	Corrupt encoding in the DB	Sample skipped, warning logged
T17	Webcam unplugged or blocked in Chrome	Clear camera error message

Measuring accuracy. I will enroll N people and run 10 scans each. Then I report the correct-match rate, false rejections, and false accepts at my chosen threshold. This is strong evidence for the technical defense.

Automation. I will automate T04 to T07, T09, T10, and T13 with pytest, using files from sample_images/.

---

12. Privacy, Ethics, and Security

- I include an explicit consent checkbox and store the consent date for each person.
- I store encodings, not photos. Photos are processed in memory and discarded.
- I only use myself and classmates who agreed.
- I provide a "Delete my data" function (cascade delete from persons).
- In the Philippine context, this relates to the **Data Privacy Act of 2012** (Republic Act No. 10173). Biometric data is sensitive personal information, which is why I limit what I store.
- Known limitation: no liveness detection, so a photo may fool it. Future work: blink detection or challenge-response.
- For real deployment I would encrypt the database and restrict access.

---

13. My Work Plan and Individual Contribution

I am working on this project alone, so I am responsible for every module, the documentation, and the testing. To keep the work manageable, I will build one module at a time and test it before moving on.

Area	What I build	Deliverable
Vision	vision.py, validation.py	Detection, encoding, quality checks, threshold tuning
Data	db.py, rules.py, schema.sql	Tables, BLOB handling, duplicate and late rules
Interface	app.py, pages/, reports.py	Enroll, Attendance, and Reports pages, CSV export
Quality and documentation	tests/, diagrams, README.md	Test cases, bug log, architecture diagram, slides

Contribution_Record. Because I am the only member, my one-page record states that I completed all four areas above. I will support it with my git commit history, and I will note which classmates volunteered as test subjects.

Working alone also means I have no teammate to review my code, so I will rely on automated pytest tests and a written bug log to catch my mistakes.

---

14. Mapping to the Grading Requirements

Requirement in the guide	How my project satisfies it
Clear problem	Manual attendance is slow and falsifiable
2+ connected components	UI, vision module, rules engine, SQLite
Visible input-process-output flow	Webcam to encoding to match to database to report; IPO model in Section 4.2 and flowchart in Section 4.3
Instructor-approved data exchange	Database operations, module-to-module calls, CSV export
Validation and error messages	Section 9
Meaningful transformation	Image to encoding to match to status, plus aggregation
Usable interface	Streamlit multi-page app
Accurate diagram	Figures 1 to 3 in Section 4, redrawn to match my final code
Working final version	Live demo with enrolled classmates

15. Live Demo Plan (5 to 7 minutes)

Before the demo: charger plugged in, other apps closed, lighting tested in the demo room, backup video recorded and copied to my second SSD and a USB drive.

1. **Start state:** fresh demo database, app started with streamlit run.

2. Enroll myself and one volunteer classmate (show the consent checkbox and multi-photo capture).
3. Show a rejected enrollment (no face or two faces).
4. Scan an enrolled member: **Present**.
5. Scan again: **Duplicate** message.
6. Use the late cutoff (change it in config or use a later scan): **Late**.
7. Scan an unenrolled person: **Unknown**.
8. Open Reports: show the table, chart, and Absent list, then download the CSV.
9. Run sqlite3 attendance.db "SELECT * FROM attendance;" to show the stored result.

---

16. Defense Question Preparation

Question	What I will say
Main components and why separated?	Section 4.4: testability, swappability, role ownership
How does data move?	Section 6, the IPO model (Figure 2), and the architecture diagram (Figure 1)
What format is exchanged?	Section 7: RGB arrays, 128-d vectors, 1024-byte BLOBs
Validation and transformation?	Sections 8 and 9
Invalid input or unavailable resource?	The error table; the app never crashes
Hardest part?	Installing dlib on Python 3.12, tuning the threshold, and consistent lighting
How did testing confirm it works?	Test cases T01 to T17 and the accuracy measurement
What would I improve?	Liveness detection, encrypted DB, FAISS index, admin login, REST API
Individual contributions?	I did the whole project alone: Section 13 and my git history show every module
Why no GPU or CNN model?	My laptop has only integrated Intel graphics, so I chose the CPU HOG detector, which is accurate enough for a front-facing webcam

17. Common Problems and My Fixes

Problem	Fix
dlib build fails	Confirm cmake, build-essential, and python3-dev are installed; otherwise use YuNet/SFace
No module named 'pkg_resources'	pip install "setuptools<81" inside the venv
NumPy compatibility error	pip install "numpy<2" and reinstall opencv-python
Webcam not found	ls /dev/video*, check the video group, make sure Cheese or another app is not holding the camera
Chrome does not show the camera	Allow the camera for localhost in the site settings
Slow recognition	Downscale frames, use model="hog", avoid the CNN model
Laptop gets hot or throttles	Hard surface, plugged in, close other apps

Problem	Fix
False matches	Lower the threshold toward 0.5; enroll more varied samples
Missed matches in another room	Enroll under similar lighting, add samples from several conditions
SQLite “database is locked”	Short-lived connections and sqlite3.connect(path, timeout=10)
Streamlit reruns on every click	Keep state in st.session_state; cache carefully

18. My Build Timeline

1. **Days 1 to 2:** install the stack, verify the webcam, detect a face in a still photo.
2. **Days 3 to 4:** database schema and a BLOB save/load round-trip test.
3. **Days 5 to 6:** enrollment page with validation.
4. **Days 7 to 8:** attendance page with matching and rules.
5. **Days 9 to 10:** reports, charts, CSV.
6. **Days 11 to 12:** testing, bug fixing, threshold tuning, accuracy measurement.
7. **Days 13 to 14:** diagrams, documentation, slides, rehearsal, backup video.

---

19. References

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)* (Vol. 1, pp. 886-893). IEEE. https://doi.org/10.1109/CVPR.2005.177

Data Privacy Act of 2012, Rep. Act No. 10173 (2012). https://www.officialgazette.gov.ph/2012/08/15/republic-act-no-10173/

Geitgey, A. (2020). *face_recognition* (Version 1.3.0) [Computer software]. GitHub. https://github.com/ageitgey/face_recognition

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Rio, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770-778). IEEE. https://doi.org/10.1109/CVPR.2016.90

Hipp, D. R. (n.d.). *SQLite documentation*. SQLite. https://www.sqlite.org/docs.html

King, D. E. (2009). Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research, 10*, 1755-1758.

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56-61). https://doi.org/10.25080/Majora-92bf1922-00a

OpenCV. (n.d.). *OpenCV documentation*. https://docs.opencv.org

Python Software Foundation. (n.d.). *sqlite3: DB-API 2.0 interface for SQLite databases*. Python documentation. https://docs.python.org/3/library/sqlite3.html

Streamlit Inc. (n.d.). *Streamlit documentation*. https://docs.streamlit.io